

Real-Time Incident Management Dashboard

Final Report for the Distributed Systems Course 2025/2026

Alessandro Martini

`alessandro.martini10@studio.unibo.it`

July 23, 2026

This project presents a distributed real-time incident management system designed for collaborative ticket resolution in operational environments. The system enables multiple operators to simultaneously interact with a shared pool of incidents through a web-based Kanban dashboard, providing real-time synchronization, mutual exclusion for ticket claiming, and automatic SLA-based escalation. The architecture follows a three-tier pattern: a Vue.js 3 single-page application serves as the presentation layer, an Express.js API with Socket.IO handles real-time communication and business logic, while a MongoDB Replica Set ensures persistent strongly-consistent state and Redis provides distributed coordination. Key distributed systems patterns implemented include: (i) Redis-based distributed locking with Lua scripts for atomic claim/release operations, preventing race conditions during ticket assignment; (ii) optimistic locking via document versioning in MongoDB to avoid lost updates during concurrent modifications; (iii) leader election using Redis TTL keys for exclusive execution of background tasks (automatic escalation of stale incidents); (iv) MongoDB Change Streams for reactive, event-driven synchronization across all connected clients without polling. The entire system is containerized and orchestrated via Kubernetes, featuring a 3-node MongoDB StatefulSet for data persistence and failover, Horizontal Pod Autoscaler for the API tier (3–10 replicas), and an Ingress with session affinity. The CAP theorem is addressed by choosing CP consistency for the data layer (MongoDB) while providing graceful AP degradation for coordination features (Redis), ensuring critical incident data remains consistent even during network partitions.

Quick L^AT_EX suggestions

If you need to cite a reference, you can use the `cite` command, like providing the BibTeX key of some entry in the `references.bib` file, e.g.: [1].

You can find pre-cooked BibTeX entries for most Computer Science papers on DBLP.



Figure 1: This is an example image

If you need to include an image, let $\text{\LaTeX}$ decide where to put it by using the `figure` environment, with a `includegraphics` command inside it. If you need to reference a figure, use the `label` command to assign a label to the figure, and then use the `cref` command to reference it. Put figures in the `figures` folder. A complete example is shown in fig. 1.

Do **not** put any placement constraint on figures, such as `[h]` or `[h!]`. Same considerations apply for other floating elements (e.g. tables, algorithms, listings, etc.).

Do **not** use the `\\` or `\newline` commands to break lines.

Just leave an empty line between two paragraphs to start a new one. The new line will be automatically indented. This is intended: it is the $\text{\LaTeX}$ way to separate paragraphs.

1 Concept

1.1 Product Type

The project consists of a **web application** with a real-time graphical user interface (GUI). Specifically, it is a *Single-Page Application* (SPA) built with Vue.js 3, served as a static web application and interacting with a distributed backend via HTTP and WebSocket protocols. The frontend presents a **Kanban-style dashboard** that allows multiple operators to visualize, create, claim, and resolve incidents in real time.

1.2 Use Case Description

1.2.1 What the software does

The system implements a **real-time incident management board** (similar to a ticketing or ITSM system) with the following core capabilities:

- **Incident registration:** operators can create new incidents with a title and initial status open

- **Ticket claiming:** an operator can *lock* an incident for exclusive processing, preventing other operators from simultaneously working on the same ticket (mutual exclusion)
- **Ticket resolution:** the claiming operator can mark an incident as **closed**, releasing the lock and notifying all connected clients
- **Automatic escalation:** a background task periodically scans for incidents that have remained **open** for more than 5 minutes and automatically escalates them to **escalated** status, signaling SLA breach
- **Real-time synchronization:** all connected clients receive live updates whenever an incident is created, modified, locked, unlocked, or escalated, without manual page refresh
- **User presence tracking:** the system displays the list of currently connected operators

1.2.2 Where the users are

Users (operators) access the system from **web browsers** connected to the same network where the Kubernetes cluster is deployed. The Ingress controller exposes the application at `http://board.local`, making it accessible to any client within the network reach. Given the containerized deployment on Kubernetes, the system could be deployed on any cloud provider or on-premises infrastructure, making the operators geographically distributed as long as they have network connectivity to the cluster.

1.2.3 When and how frequently they interact

Interaction is **continuous and event-driven** during operational shifts. Operators may:

- Create incidents sporadically (when new issues are reported)
- Claim and resolve incidents in bursts (during active troubleshooting)
- Monitor the dashboard passively (watching for escalations or new tickets)

The background escalation task runs every 10 seconds, making the system inherently time-sensitive.

1.2.4 How they interact and which devices

Users interact through a **desktop web browser** (the responsive design supports any modern browser). The interaction model is:

- **Login phase:** the operator enters their username to join the board
- **Dashboard phase:** a Kanban board displays four columns (Open, In Progress, Escalated, Resolved) with draggable cards

- **Actions:** buttons for claiming, releasing, and resolving tickets; a search bar for filtering; toggle buttons for viewing only escalated or personal tickets
- **Notifications:** toast notifications alert the operator of events (new tickets, escalations, resolutions by others)

1.2.5 Data storage

The system stores the following data persistently in **MongoDB**:

- **Incidents:** title, status (`open`, `escalated`, `closed`), creator (`createdBy`), closer (`closedBy`), version number for optimistic locking, and timestamps
- **Version history:** implicitly tracked via the `version` field and `updatedAt` timestamps

Redis is used for **transient coordination state** (not persistent data):

- Distributed locks for ticket claiming (keys with TTL)
- Leader election key for the background escalation task
- `Socket.IO` adapter for cross-pod message broadcasting

1.2.6 User roles

The system currently supports a **single role** (operator), but the architecture naturally supports differentiation:

- **Operator:** can create, claim, release, and resolve incidents; views all incidents and online users
- (Extensible) **Supervisor:** could be added to view analytics, override locks, or manage SLA thresholds

1.3 Why Distribution is Needed

Distribution is not merely an implementation choice but a **fundamental requirement** for this system, motivated by the following factors:

1.3.1 Fault Tolerance

The system must remain operational even if individual components fail. With a single-node deployment, the failure of the API server or the database would render the entire ticketing system unusable—a critical risk in operational environments where incident tracking cannot afford downtime. The Kubernetes architecture addresses this by:

- Running **3 API replicas** minimum, so the loss of one pod does not interrupt service

- Deploying MongoDB as a **3-node Replica Set**, which tolerates the failure of one node while maintaining a primary for writes
- Using Redis for shared state, so any API pod can take over coordination tasks

1.3.2 Scalability

As the number of concurrent operators and incidents grows, a single API instance would become a bottleneck. The system employs a **Horizontal Pod Autoscaler** (HPA) that scales the API tier from 3 to 10 replicas based on CPU utilization, ensuring that:

- HTTP request handling capacity increases with load
- Socket.IO connections are distributed across multiple pods (via the Redis adapter)
- The background escalation task is executed by exactly one leader, regardless of pod count

1.3.3 Resource Sharing

Multiple operators must access and modify the **same shared pool of incidents** concurrently. Without distribution, the system would be limited to a single operator at a time, defeating its collaborative purpose. The distributed architecture enables:

- **Concurrent read access:** all operators see the same live state of the Kanban board
- **Coordinated write access:** distributed locks ensure that only one operator can claim a ticket at a time, while optimistic locking prevents conflicting updates

1.3.4 Geographic Distribution (Potential)

While the current deployment targets a single Kubernetes cluster (e.g., Minikube or a single cloud region), the architecture is designed to support geographic distribution. The MongoDB Replica Set could be extended with nodes in different data centers, and the API tier could be replicated across regions behind a global load balancer, enabling operators in different locations to collaborate on the same incident pool.

2 Requirements Elicitation and Analysis

This chapter defines *what* the system must do (functional requirements), the qualities it must exhibit (non-functional requirements), and the constraints imposed on its realization (implementation requirements). Each requirement is numbered and paired with an acceptance criterion that is later used, unchanged, as the basis for the validation phase (section 5).

2.1 Functional Requirements

- 1 Board access.** An operator can join the board by choosing a username, without prior registration or authentication.
Acceptance criterion: submitting a non-empty username transitions the client from the login screen to the dashboard and establishes a persistent WebSocket connection.
- 2 Incident creation.** An operator can create a new incident by providing a title; the system assigns it a default status of **open**, a creation timestamp, and the creator's identity.
Acceptance criterion: a valid `POST /api/incidents` request returns 201 and the new incident appears as a card in the "Open" column of every connected client without any client issuing a manual refresh.
- 3 Real-time board synchronization.** All connected operators must observe incident creations, claims, releases, resolutions, and escalations as they happen.
Acceptance criterion: an action performed by operator A becomes visible on operator B's board without B reloading the page or polling.
- 4 Ticket claiming (mutual exclusion).** An operator can claim an **open** or **escalated** incident for exclusive handling; once claimed, no other operator can claim or resolve it.
Acceptance criterion: of two concurrent claim requests on the same incident, exactly one succeeds (200) and the other is rejected (409, reporting the current owner).
- 5 Explicit claim release.** An operator holding a claim can release it without resolving the incident, returning it to the shared pool for other operators.
Acceptance criterion: after a successful release, the incident becomes claimable again immediately, without waiting for the automatic lock expiry.
- 6 Ticket resolution.** The operator currently holding the claim on an incident can mark it as **closed**, recording who closed it and releasing the associated lock.
Acceptance criterion: a resolved incident moves to the "Resolved" column on every client and can no longer be claimed or resolved again.
- 7 Automatic SLA escalation.** Incidents left in the **open** status beyond a configured threshold (5 minutes) are automatically transitioned to **escalated** without operator intervention.
Acceptance criterion: an incident older than the threshold changes to **escalated** within one execution cycle of the background task (≤ 10 seconds after threshold crossing) and moves to the "Escalated" column on every client.
- 8 Presence awareness.** The system displays the set of operators currently connected to the board.
Acceptance criterion: the online-users list reflects a new connection within one

broadcast cycle, and a user disappears from the list once their connection is terminated (including ungraceful terminations).

- 9 Filtering and search.** An operator can narrow the visible incidents via free-text search on the title, an “escalated only” toggle, and a “mine only” toggle (incidents they created, claimed, or resolved).

Acceptance criterion: activating any filter combination updates only the Kanban columns; the KPI summary counters continue to reflect the true, unfiltered system state.

- 10 Event notifications.** Operators receive non-blocking toast notifications for relevant events they did not personally trigger, and for local operation failures.

Acceptance criterion: every failed claim, release, or resolve action produces a visible notification describing the reason for failure; no user-initiated action fails silently.

2.2 Non-Functional Requirements

- 1 No lost updates.** Concurrent modification attempts on the same incident must never silently overwrite one another.

Realized by: optimistic locking via a `version` field, matched on every `PATCH`.

Acceptance criterion: a `PATCH` carrying a stale version is rejected with `409` and leaves the document unmodified.

- 2 Server-enforced mutual exclusion.** The claim mechanism must be enforced by the server, not merely suggested by the client interface.

Acceptance criterion: a resolution request issued by an operator who does not currently hold the incident’s lock is rejected with `409`, even when the request bypasses the UI entirely (e.g., sent directly to the API).

- 3 Fault tolerance of the API tier.** The loss of any single API pod must not interrupt service for connected or new operators.

Acceptance criterion: deleting one of the ≥ 3 running `api` pods produces no visible downtime; remaining pods continue serving requests, and `Socket.IO` clients transparently reconnect to a healthy pod.

- 4 Fault tolerance of the data tier.** The loss of any single MongoDB node must not interrupt read/write availability nor lose committed data.

Realized by: a 3-node Replica Set with automatic primary election, protected by a `PodDisruptionBudget` (`minAvailable: 2`) that prevents voluntary evictions from breaking quorum.

Acceptance criterion: terminating one of the 3 `mongo` pods triggers automatic recovery (re-election if the primary was lost) without manual intervention, and the API resumes normal operation once the probe reports readiness again.

- 5 Exactly-once background execution.** The automatic escalation task must execute exactly once per cycle across the whole cluster, regardless of how many

API pods are running.

Realized by: Redis-based leader election with an atomically-renewed lock (Lua script, preventing the split-brain window of a naive `GET+PEXPIRE`).

Acceptance criterion: with $N \geq 1$ running `api` pods, a given incident is escalated exactly once (no duplicate version increments, no duplicate log entries).

- 6 Horizontal scalability.** The API tier must absorb increased load by adding replicas automatically, without manual intervention.

Realized by: a `HorizontalPodAutoscaler` (3–10 replicas, 80% CPU utilization target).

Acceptance criterion: sustained CPU load above the threshold results in additional `api` pods being scheduled within the HPA’s stabilization window.

- 7 Reconnection consistency.** A client that experiences a temporary disconnection (e.g., during a rolling update of the API tier) must never regress to a state older than what it displayed before disconnecting.

Realized by: a connection-generation counter that discards stale resynchronization responses arriving after a newer reconnection has already occurred.

Acceptance criterion: rapid, repeated reconnects never leave the client displaying data fetched by an earlier, superseded connection.

- 8 Self-healing presence data.** The count of online operators must never permanently overcount stale sessions after an ungraceful pod termination.

Realized by: deriving presence from `io.fetchSockets()` (live socket introspection across the Redis adapter) instead of a manually-maintained Redis hash, which was found during development to leak stale entries on unclean shutdowns.

Acceptance criterion: after force-terminating an API pod, the online-user count converges to the correct value without manual state cleanup.

- 9 UI eventual consistency.** A given incident must never be rendered in two mutually-exclusive Kanban columns at once, regardless of the arrival order of independent real-time events describing the same state change.

Acceptance criterion: resolving a claimed incident never produces a transient duplicate card appearing simultaneously in “In Progress” and “Resolved”, irrespective of whether the lock-release event or the database change-stream event is delivered to the client first.

2.3 Implementation Requirements

- 1 The backend is implemented in Node.js, using Express for the REST API and Socket.IO for the WebSocket transport. This follows from the design need for a single event-driven runtime capable of hosting both request/response handling and long-lived WebSocket connections in the same process, which simplifies bridging MongoDB Change Stream events onto Socket.IO broadcasts.

- 2 MongoDB is deployed as a genuine multi-node Replica Set, not a single-member replica set masquerading as one. A one-member replica set would be sufficient to unlock the Change Streams API, but would not exercise the replication and automatic-failover mechanisms.
- 3 Redis is present as the shared coordination point for all cross-instance mechanisms (Socket.IO Pub/Sub adapter, distributed locks, leader election). This follows directly from the design decision to keep the API tier stateless and horizontally scalable, which in turn requires an external, shared point of coordination reachable by every replica.
- 4 The system is containerized with Docker and orchestrated with Kubernetes, validated locally on Minikube. This constraint follows from the pedagogical objective of exercising orchestration-level distributed-systems mechanisms (StatefulSets, PodDisruptionBudgets, Horizontal Pod Autoscaling, liveness/readiness separation) that would not otherwise be exercised by application code alone.
- 5 The frontend is a Vue 3 single-page application using the Composition API. This is a project-specific implementation choice rather than an externally imposed constraint: it emerged as a consequence of the design's reliance on fine-grained reactive state derived from a continuous stream of WebSocket events.

2.4 Glossary

- **Claim:** the exclusive right to work on an incident, granted through a distributed lock (Redis SET NX PX) with a bounded time-to-live.
- **Escalation:** the automatic transition of an incident from `open` to `escalated` once it has exceeded the configured SLA threshold without being resolved.
- **Leader:** the single pod, among all running replicas of the `api` Deployment, elected at any given time to execute a shared background task (here, the escalation scan).
- **Change Stream:** a MongoDB feature allowing a client to subscribe to a real-time feed of changes on a collection, implemented on top of the operation log (oplog) and available only on a Replica Set.
- **Optimistic locking:** a concurrency-control strategy that allows unrestricted concurrent reads and detects conflicting writes at commit time via a version check, rather than serializing access up front.
- **Readiness / Liveness probe:** Kubernetes-level health checks that determine, respectively, whether a pod should receive traffic and whether it should be restarted.
- **Sticky session:** a load-balancing strategy, configured on the Ingress via a cookie, that routes all requests belonging to one client session to the same backend replica.

- **PDB (PodDisruptionBudget):** a Kubernetes resource limiting how many replicas of a workload may be voluntarily disrupted (e.g., during a drain or rolling update) at the same time.

2.5 Relevant Distributed System Features

2.5.1 Transparency

Transparency is **partially relevant** to this system, and deliberately *not* pursued in full. Location and replication transparency are provided where they reduce accidental complexity: an operator’s browser never needs to know which of the (3 to 10) `api` pods it is talking to, nor which of the 3 `mongo` replicas served a given read—both are hidden behind, respectively, the Ingress with sticky sessions and the replica-set connection string. Failure transparency is provided only up to a point: a client experiencing a pod failure automatically reconnects and resynchronizes rather than surfacing a raw connection error. Concurrency transparency, however, is *intentionally not* pursued: when an incident is claimed, the system deliberately exposes this fact to other operators (via the lock indicator and disabled actions) rather than hiding it, because SOC analysts specifically need to know that a colleague is already handling a given incident to avoid duplicated effort.

2.5.2 Fault Tolerance, Dependability

The system is required to survive the loss of any single component: any one `api` pod (stateless, backed by ≥ 3 replicas), any one `mongo` node (backed by a 3-node Replica Set and a quorum-preserving `PodDisruptionBudget`), and transient unavailability of Redis (which is coordination-only state, not the system of record). Data loss or silent corruption is explicitly unacceptable, which motivated optimistic locking over a “last write wins” strategy. Recovery is also fast: the leader-election lock has a 15-second time-to-live specifically so that the background escalation task resumes within seconds of a leader’s failure, rather than requiring manual restart. One honestly-documented limitation, discovered during development rather than assumed by design, is that Redis is currently a *single, non-replicated* instance: it is a deliberate simplification appropriate to the scope of this project, and is discussed further as a known limitation in section 9.

2.5.3 Scalability

Scalability is **relevant** for the stateless API tier and **explicitly out of scope** for the data and coordination tiers. The number of concurrent operators and the request rate they generate can grow, which is why the `api` Deployment is fronted by a `HorizontalPodAutoscaler` scaling between 3 and 10 replicas on CPU utilization. MongoDB and Redis, by contrast, are scaled for *availability* (the former) or not scaled at all (the latter): the Replica Set’s 3 nodes exist to tolerate failure, not to increase read/write throughput, and no read-preference tuning is applied to route reads to secondaries. This asymmetry is a conscious trade-off given the project’s scope, not an oversight.

2.5.4 Security, Trust

Security is **deliberately minimized** in the current iteration, in favor of the project’s primary focus on distributed coordination. There is no authentication: any operator can join the board with an arbitrary, unverified username, and there is a single implicit role with no differentiated permissions. No transport encryption is configured between internal components (API, MongoDB, Redis all communicate over the cluster-internal network without TLS), and no data is encrypted at rest. These omissions are acceptable for the current scope—the system is not intended to handle sensitive data beyond incident titles and operator names—but are explicitly flagged as a direction for future work (section 9) rather than an oversight not considered during analysis.

2.5.5 Resource Sharing

Resource sharing is the **central motivation** for the entire project: multiple SOC operators must concurrently read and modify the *same* shared pool of incidents, which is precisely the scenario that requires distributed coordination rather than a simple single-user application. This single feature justifies two of the three coordination mechanisms implemented: the distributed lock, which serializes exclusive access to an individual incident (claim/release), and optimistic locking, which detects (rather than prevents) conflicting concurrent writes on the shared MongoDB collection.

2.5.6 Openness, Interoperability, Heterogeneity of Components

This feature is **not relevant** to the current scope. The system is a closed application that does not interact with external ticketing or monitoring systems, and does not expose a public, versioned API intended for third-party consumption. The architecture does not preclude such integration in the future (the REST layer could, in principle, be consumed by an external system), but no standardization or interoperability requirement was elicited for this iteration.

2.5.7 Evolvability, Maintainability

This feature is **relevant**, and was reinforced by direct experience during development. The codebase is organized into three independently deployable units (`api/`, `web/`, `k8s/`), and the liveness/readiness probe separation (shallow vs. deep health checks) exists specifically to allow rolling updates of the API tier without unnecessary restarts or traffic loss. At the same time, maintaining hand-rolled Redis primitives (the distributed lock and the leader-election lock) proved to carry real maintenance cost: an initial, non-atomic `GET+PEXPIRE` renewal of the leadership lock was later replaced with an atomic Lua script after a split-brain risk was identified, and a manually-maintained “active users” Redis hash was later replaced with `io.fetchSockets()` after it was observed to leak stale entries across unclean pod restarts. These are documented here as evidence that DIY coordination primitives require deliberate care to remain correct as the system evolves, not merely to be implemented once.

2.5.8 Performance, Concurrency, Computation/Communication Efficiency, Bandwidth

This feature is **relevant** but not subject to strict quantitative requirements (e.g., no fixed latency SLA was elicited). Real-time propagation relies on MongoDB Change Streams rather than polling, and on Socket.IO over WebSocket rather than repeated HTTP requests, both chosen for communication efficiency. One trade-off surfaced during design is that every `api` pod opens its own independent Change Stream cursor against MongoDB; this keeps the cross-pod broadcast logic simple (each pod emits only to its own locally-connected clients, avoiding duplicate delivery through the Redis adapter) at the cost of the primary node serving one concurrent change-stream cursor per pod, a cost that grows linearly as the HPA scales the API tier toward its 10-replica maximum.

2.5.9 Economy, Costs

This feature is **not a primary driver** of the design, since the project targets a local Minikube cluster for development and academic validation rather than a production deployment with a defined operating budget. Where cost-conscious choices were made, they follow from minimizing complexity rather than minimizing a monetary budget: for instance, Redis is kept as a single instance rather than a Sentinel/Cluster deployment, and MongoDB uses the minimum replica count (3) that still permits a majority-based quorum, rather than a larger cluster.

3 Design

3.1 Architecture

The system follows a **three-tier, layered architecture** (presentation, application, data), combined with an **event-driven / publish-subscribe** style for state propagation across tiers. It is deliberately *not* decomposed into multiple microservices: a single stateless application service (`api`) hosts both the REST endpoints and the WebSocket server, since the two share the same domain logic and splitting them would only introduce inter-service coordination overhead without a corresponding benefit at this project's scale.

Two motivations drive this style, both traceable to section 2:

- **Horizontal scalability of the stateless tier:** a layered style with a stateless application layer is what makes it possible for the `api` Deployment to be scaled elastically (3–10 replicas via the HPA) without any replica needing to know about the others' in-memory state — all shared state that must survive a request lives either in MongoDB (durable) or Redis (ephemeral coordination), never in a pod's local memory.
- **Near-real-time propagation:** a purely request/response (RPC-style) architecture would force clients to poll for updates. The event-driven style — MongoDB

Change Streams triggering Socket.IO broadcasts — allows the data layer itself to be the source of truth for *what changed*, decoupling “a write happened” from “who needs to know about it.”

3.2 Infrastructure

The infrastructure is composed of the following components, all containerized and orchestrated by Kubernetes (validated on Minikube):

- **Clients:** operator web browsers, running the Vue.js 3 SPA.
- **Static file server:** an Nginx container (**web** Deployment) serving the compiled Vue.js bundle; stateless, single replica.
- **Application servers:** Express.js + Socket.IO processes (**api** Deployment), stateless, horizontally scaled 3–10 replicas by a **HorizontalPodAutoscaler** on CPU utilization.
- **Reverse proxy / load balancer:** the Kubernetes Ingress (Nginx controller, enabled as a Minikube addon), which performs path-based routing (/ to **web**, /api/* and /socket.io/* to **api**) and cookie-based sticky sessions across the **api** replicas.
- **Database:** a 3-node MongoDB Replica Set (**StatefulSet**), the durable system of record for incidents.
- **Message broker / coordination store:** a single Redis instance, serving three distinct roles simultaneously — the Socket.IO Pub/Sub adapter (cross-pod Web-Socket broadcast), the distributed-lock store (incident claims), and the leader-election store (background task).
- **Batch job:** a one-shot Kubernetes Job that initializes the MongoDB Replica Set exactly once, decoupled from the **StatefulSet**’s own lifecycle.

Network placement. In the validated deployment, all components run within a single Kubernetes cluster (a single-node Minikube instance), but each is an independently scheduled Pod communicating exclusively over the cluster’s internal network — nothing in the design assumes co-location on the same physical node. The architecture therefore does not preclude spreading these components across multiple nodes, or even multiple datacenters, in a production-grade cluster (ex, *Geographic Distribution (Potential)*); this was intentionally left as a deployment-time concern rather than an application-level one.

Component discovery. All discovery is handled natively by Kubernetes rather than by any custom mechanism in the application code:

- Stateless components (**api**, **web**, **redis**) are addressed through regular **ClusterIP** Services and Kubernetes’ internal DNS (e.g., **redis.app.svc.cluster.local**), which also perform basic load balancing across their pods.

- The MongoDB Replica Set uses a *headless* Service (`mongo.app.svc.cluster.local`) paired with the `StatefulSet`'s stable per-pod DNS names (`mongo-0.mongo`, `mongo-1.mongo`, `mongo-2.mongo`), since replica-set members must be individually addressable rather than load-balanced — clients (and members electing a primary) need to reach a *specific* node, not *any* node.
- Externally, the Ingress performs name-based virtual hosting (`board.local`) and path-based routing, so the only name an operator's browser ever needs to know is a single hostname.

3.3 Modelling

Domain entities. The core domain entity is the **Incident** (`title`, `status` $\in \{\text{open}, \text{escalated}, \text{closed}\}$, `createdBy`, `closedBy`, `version`, `updatedAt`). Two further concepts exist but are deliberately *not* modelled as persistent entities: the **Operator** (identified only by a self-declared username, never stored beyond the lifetime of a WebSocket connection) and the **Claim** (the exclusive right to work on an incident, which exists only as a key with a time-to-live in Redis, never written to MongoDB).

Mapping to infrastructure. This distinction directly reflects each entity's durability requirement:

- The **Incident** must survive pod restarts, rolling updates, and node failures, and is therefore stored in MongoDB, replicated across the 3-node Replica Set.
- The **Claim** and the **leadership mandate** are, by design, short-lived and self-expiring; persisting them durably would be both unnecessary and actively harmful (a durable lock that outlives its holder's crashed connection would deadlock the resource). They are therefore stored in Redis, an in-memory store with native TTL support.
- The **Operator**'s identity is not persisted at all: it exists only as `socket.data.username` on a live connection, discoverable cluster-wide on demand via `io.fetchSockets()`, and simply ceases to exist the moment the connection closes.

Domain events. `IncidentCreated`, `IncidentClaimed`, `IncidentReleased`, `IncidentResolved`, `IncidentEscalated`, and `PresenceChanged` are the events that drive the system. They are realized, respectively, as: MongoDB Change Stream notifications for the events that mutate the Incident's persistent state (created, resolved, escalated), and direct Socket.IO broadcasts for the events that mutate only ephemeral Redis state (claimed, released, presence changed) — see section 3.4 for why these two categories are propagated through different channels.

Messages. Two distinct message categories are exchanged: **commands** (client → server, over REST: `POST /incidents`, `POST/DELETE .../claim`, `PATCH /incidents/:id`), which expect a definite success/failure reply and change system state; and **events** (server → client, over Socket.IO: `incident_update`, `incident_locked/incident_unlocked`, `users_update`), which are fire-and-forget notifications with no expected reply. There are no *queries* beyond the initial `GET /api/incidents` used once at connection time (and again after every reconnect) to hydrate a client's local state — after hydration, the client never polls again, relying entirely on the event stream to stay current.

System state. The authoritative state of the system is split across two stores by design: MongoDB holds the durable Incident collection; Redis holds transient coordination state (locks, leadership). Each connected client additionally holds a *derived*, client-local copy of a filtered view of that state (the `incidents` and `onlineUsers` reactive references in the Vue application), which is not authoritative and is kept eventually consistent with the server through the event stream and the reconnect-time resynchronization.

3.4 Interaction

Two interaction patterns coexist, cleanly separated by direction:

- **Client → server: request/response.** Every mutation (create, claim, release, resolve) is a synchronous HTTP call that yields a definite outcome (`2xx` success or `4xx/5xx` failure) before the client considers the action complete. This pattern is used specifically because these operations need an unambiguous, immediate result (e.g., “did I get the lock or not?”), which a fire-and-forget message could not provide as directly.
- **Server → client: publish/subscribe.** All state changes are broadcast to every interested client without any client explicitly requesting them, and without the server tracking who has or hasn't yet seen a given update. No server-initiated request ever targets one specific client.

Within the publish/subscribe direction, two further sub-patterns are used depending on the origin of the change, both of which were the subject of extensive tuning during development (section 2.2):

- Changes to **persistent** state (create, resolve, escalate) are captured by MongoDB Change Streams and broadcast via `io.local.emit()` — *not* `io.emit()` — because every `api` pod independently watches the same Change Stream and would otherwise re-broadcast an event that every other pod has *already* independently received and forwarded to its own clients, causing duplicate delivery.
- Changes to **ephemeral** state not reflected in MongoDB (claim, release, presence) are broadcast via `io.emit()`, which *does* need to cross pod boundaries through the Redis adapter, since only the one pod that handled the originating REST request knows the change happened.

3.5 Behaviour

- **api pods** are stateless with respect to domain data: nothing about an incident, a claim, or a user is cached in a pod's own memory across requests. Each pod reacts to three independent stimuli: (1) incoming REST requests, handled synchronously; (2) events pushed by its own MongoDB Change Stream subscription, handled asynchronously and forwarded only to its own locally-connected sockets; (3) an internal timer (every 10 seconds) that drives both the leader-election attempt and, if leadership is held, the escalation scan — this third stimulus is self-generated, not triggered by any external message.
- **MongoDB** accepts writes only on the elected primary, which replicates them to the two secondaries via the oplog; if the primary becomes unreachable, the remaining nodes run an automatic election (majority-based) to promote a new primary, with no application-level intervention required.
- **Redis** is purely reactive and carries no domain logic of its own: it only executes the commands (`SET NX PX`, `EVAL` of the Lua scripts, `GET`, `DEL`) issued by the **api** pods, and expires keys autonomously once their TTL elapses.
- **The Vue.js client** does not update its local state optimistically on user action; it waits for the server's response (for the direct outcome of its own command) and for the corresponding broadcast event (for the canonical, server-confirmed state), which keeps the client's view a strict function of what the server has actually confirmed, rather than a prediction that could be wrong and require a visible correction later.

State-update ownership follows directly from section 3.3: MongoDB is the sole writer of durable Incident state (through the **api** pods, which never bypass it), Redis is the sole holder of ephemeral coordination state, and no component other than Socket.IO's own connection table is ever treated as authoritative for who is currently online.

3.6 Data and Consistency Issues

What is stored, and where. Incidents are the only data that must persist beyond a single request or connection; they are stored in **MongoDB** as documents. The document model was chosen because incidents are naturally semi-structured records whose schema evolved incrementally over the project's development (e.g., the `escalated` status and the `closedBy` field were both introduced after the initial design) without requiring a formal migration step — a flexibility that a rigid relational schema would not have offered as cheaply at this project's scale. Coordination data (locks, leadership) is stored in **Redis** as plain key-value pairs, chosen because the access pattern is a pure read/write/expire on independent keys with no relational structure, and because lock contention needs sub-millisecond latency that an in-memory store provides naturally.

Query and write patterns. `api` pods are the only components that ever query or write MongoDB and Redis; the client never talks to either data store directly, always mediating through the REST/Socket.IO layer. Concurrent *reads* are unrestricted and require no coordination (both MongoDB and Redis handle concurrent reads natively). Concurrent *writes* are expected and must be explicitly coordinated: this is precisely the motivation behind the two concurrency-control mechanisms of section 2.2 — optimistic locking detects conflicting concurrent writes to the *same* Incident document, and the distributed lock prevents two operators from concurrently claiming the same Incident in the first place.

Shared data. The Incidents collection is, by design, shared read/write state across every `api` replica — this sharing is the entire point of keeping the application tier stateless and horizontally scalable. The same is true of the Redis keyspace: coordination only works if every replica observes the same lock and leadership state, which is why a single shared Redis instance (rather than one per pod) is a hard requirement, not an implementation convenience.

Consistency model. The system provides **strong consistency** for writes to the primary (a successful write is immediately visible to any subsequent read against the primary) and **eventual consistency** for propagation to connected clients: a change is always durably committed to MongoDB *before* any client is notified of it, but different clients may observe the notification at slightly different times depending on Change-Stream and network latency. This asymmetry is a deliberate, accepted trade-off: the domain (a SOC dashboard) tolerates sub-second staleness in what operators *see*, but never tolerates staleness or loss in what is actually *recorded* as the incident’s state.

3.7 Fault-Tolerance

Replication. MongoDB is replicated across a 3-node Replica Set specifically to survive the loss of one node while preserving both durability and write availability. Redis, in contrast, is *not* replicated: it runs as a single instance, a deliberate and explicitly documented simplification (section 2.5, *Fault Tolerance, Dependability*) rather than an oversight, justified by Redis holding only re-derivable coordination state rather than the system’s durable record. The `api` tier is “replicated” in a different sense: its 3–10 pods are interchangeable stateless workers, not holders of partitioned or replicated data.

Heartbeats, timeouts, and retries. Several independent mechanisms play this role at different layers:

- Kubernetes **liveness** (`/api/health`) and **readiness** (`/api/ready`) probes act as an external heartbeat on each `api` pod and its dependencies, respectively — deliberately kept shallow and deep, so a dependency outage only removes a pod from traffic (readiness) rather than restarting a healthy process (liveness).

- Redis-backed locks (claim, leadership) use a **TTL as an implicit timeout**: rather than requiring an explicit failure detector to notice a crashed lock holder, the lock simply expires and becomes acquirable again — a lease-based approach that trades a bounded worst-case delay (up to the TTL) for the absence of any active monitoring logic.
- Socket.IO’s own `engine.io` ping/pong protocol is a heartbeat between client and server that detects dead connections independently of TCP-level timeouts, which is what makes presence tracking via `io.fetchSockets()` self-correcting.
- The Socket.IO client’s automatic reconnection is a **retry** mechanism, explicitly paired with the connection-generation counter (NFR7) so that retries cannot corrupt the client’s perceived state ordering.

Error handling. Every REST handler wraps its logic in a `try/catch` block and translates failures into a specific HTTP status (409 for a lock or version conflict, 503 for a not-yet-ready dependency, 500 for an unexpected error) rather than letting the process crash. Failure to reach MongoDB or Redis *at startup* is treated as fatal (`process.exit(1)`), by design: Kubernetes then restarts the pod into a clean, fully-initialized state instead of leaving it serving traffic from a half-initialized process. On the frontend, every mutating action surfaces a visible notification on failure — including when the server’s response body is not valid JSON — a defensive behaviour added after a real failure mode was observed during development, where a stale deployed image caused a route to 404 and the resulting parse error was silently swallowed, leaving the operator with no feedback at all.

3.8 Availability

Caching. No read cache is used for Incident data: every `GET` reads MongoDB directly. This is a deliberate simplicity choice — the read volume expected from a SOC dashboard with a handful of concurrent operators does not justify the added complexity of cache invalidation. Redis, despite superficially resembling a cache, is never used to cache MongoDB reads in this system; its only roles are coordination (locks, leadership) and message brokering (the Socket.IO adapter).

Load balancing. The Ingress load-balances HTTP and WebSocket traffic across the ≥ 3 `api` replicas, using cookie-based **sticky sessions** specifically because Socket.IO’s HTTP long-polling fallback transport requires successive requests within one handshake to reach the *same* backend process; a plain stateless REST call would not need this affinity, but the shared Ingress rule applies it uniformly for simplicity. This mechanism works together with the `HorizontalPodAutoscaler`, which keeps the pool of backends elastic under load (section 2.2).

Behaviour under network partitioning. Several partition scenarios are explicitly handled, each favouring a different side of the CAP trade-off depending on what is partitioned:

- If an `api` pod loses connectivity to Redis or MongoDB, `/api/ready` starts failing and Kubernetes removes the pod from the Service’s endpoint list — it stops receiving traffic but is *not* killed, and rejoins automatically once connectivity is restored. This favours **consistency and correctness** (no pod serves requests it cannot fulfil correctly) over that individual pod’s availability.
- If a MongoDB secondary is partitioned from the primary, it simply falls behind and resynchronizes once the partition heals, without disrupting the primary’s availability.
- If the primary itself is partitioned from the majority of the Replica Set, it steps down to avoid a split-brain scenario (two nodes both believing they are primary), again favouring **consistency (CP)** over the availability of the minority side — the design explicitly accepts that the minority partition becomes unavailable for writes rather than risk divergent, unreconcilable incident data.

3.9 Security

Authentication. None is implemented: any operator can join the board with an arbitrary, unverified username. This is a scoped, deliberate omission (already justified in section 2.5, *Security, Trust*) rather than an oversight — the project’s focus is on distributed coordination, and the domain data involved (incident titles, operator names) does not warrant the added complexity of an identity provider for the current iteration.

Authorization. There is a single implicit role (“operator”) with no differentiated permissions — every connected client can create, claim, release, and resolve any incident. One narrow exception exists: possession of an incident’s Redis lock *does* function as a form of **capability-based authorization** for a single specific action (resolving or releasing that one incident): the server rejects a resolution attempt from anyone other than the current lock holder (section 2.2, NFR2), independent of any authenticated identity. This is authorization by *possession of a token* (the lock’s stored username value), not by verified identity, and is intentionally the only access-control boundary the current system enforces.

Cryptography. No cryptographic schemes are used anywhere in the system: communication between the `api` tier and MongoDB/Redis is unencrypted (cluster-internal traffic only), no token or signature verification is performed, and no data is encrypted at rest. As with authentication, this is an explicitly scoped-out concern rather than an unconsidered gap, and is listed as a candidate direction in section 9.

4 Implementation

Please report here all the implementation (technology-dependent) choices you made while implementing your design.

If you run out of time for this project, you may consider leaving some aspect unimplemented, and simply discuss how you would have implemented them. In this case, better would be to discuss unimplemented features in section 9.

- which **network protocols** to use?
 - e.g. UDP, TCP, HTTP, WebSockets, gRPC, XMPP, AMQP, MQTT, etc.
- how should *in-transit data* be **represented**?
 - e.g. JSON, XML, YAML, Protocol Buffers, etc.
- how should *databases* be **queried**?
 - e.g. SQL, NoSQL, etc.
- how should components be *authenticated*?
 - e.g. OAuth, JWT, etc.
- how should components be *authorized*?
 - e.g. RBAC, ABAC, etc.

4.1 Technological details

- any particular *framework* / *technology* being exploited goes here

5 Validation

5.1 Automatic Testing

- how were individual components **unit-tested**?
- how was communication, interaction, and/or integration among components tested?
- how to **end-to-end-test** the system?
 - e.g. production vs. test environment
- for each test specify:
 - rationale of individual tests
 - how were the test automated
 - how to run them
 - which requirement they are testing, if any

recall that *deployment automation* is commonly used to *test* the system in *production-like* environment

recall to test corner cases (crashes, errors, etc.)

5.2 Acceptance test

- did you perform any *manual* testing?
 - what did you test?
 - why wasn't it automatic?

6 Deployment

- should one install your software from scratch, how to do it?
 - provide instructions
 - provide expected outcomes
- what software should be installed on the machines to run your project? which versions?
- should one set environment variables or configuration files?
- if you're using containerization (e.g. Docker), describe how you engineered your deployment scrips (e.g. `docker-compose.yml` files)

7 User Guide

- how to use your software?
 - provide instructions
 - provide expected outcomes
 - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Discuss possible future works, improvements, extensions, optimizations, etc.

Also discuss here any unimplemented feature, and how you would have implemented it.

References

- [1] D. Adams. *The Hitchhiker's Guide to the Galaxy*. San Val, 1995.